

kodama-cpp: Cross-Validated Accuracy Maximization on CPU, CUDA, and Apple Metal

Moussa Kassim^{1,2}

MOUSSA.KASSIM@ICGEB.ORG

Martin Ocharo^{1,2}

MARTIN.OCHARO@ICGEB.ORG

Dalia Ahmed¹

DALIA.AHMED@ICGEB.ORG

Dupe Ojo¹

DUPE.OJO@ICGEB.ORG

Alessia Vignoli^{3,4}

VIGNOLI@CERM.UNIFI.IT

Leonardo Tenori^{3,4}

TENORI@CERM.UNIFI.IT

Stefano Cacciatore^{1,2}

STEFANO.CACCIATORE@ICGEB.ORG

¹*Bioinformatics Unit, International Centre for Genetic Engineering and Biotechnology (ICGEB), Cape Town 7925, South Africa*

²*Department of Integrative Biomedical Sciences, Institute of Infectious Disease & Molecular Medicine (IDM), University of Cape Town, Cape Town 7925, South Africa*

³*Department of Chemistry “Ugo Schiff”, University of Florence, Sesto Fiorentino, Italy*

⁴*Magnetic Resonance Center (CERM), University of Florence, Sesto Fiorentino, Italy*

Editor: To be assigned

Abstract

KODAMA searches for latent structure by maximizing the held-out predictability of evolving labels. We present **kodama-cpp**, a standalone C++17 implementation for multicore CPU, NVIDIA CUDA, and Apple Metal. One float32 core owns folds, labels, workspaces, and graph outputs, and exposes KNN or SIMPLS followed by latent-space LDA. Backend kernels provide neighbor search, k-means, label-aware cross-products, and reusable workspaces without runtime dependence on R, Python, FAISS, cuVS, RAFT, or Armadillo. The current standard specifies grouped proposals and degeneracy guards while retaining the classifier objective, but not identical label trajectories. Matrix/graph APIs and thin wrappers also expose PCA and UMAP/openTSNE. We validate primitive parity, end-to-end runtime, and external-label diagnostics.

Keywords: KODAMA, cross-validation, unsupervised learning, heterogeneous computing, manifold learning

1 Introduction

KODAMA treats labels as optimization variables: a labeling is useful when a classifier trained on some samples reproduces it on held-out samples. The original method used an ensemble of optimized vectors to correct dissimilarities before visualization (Cacciatore et al., 2014, 2017). Unlike cluster stability, prediction strength, or semi-supervised learning, KODAMA places held-out predictability inside label search without observed labels as anchors (Ben-Hur et al., 2002; Tibshirani and Walther, 2005; Chapelle et al., 2006).

Repeated fitting makes KODAMA a systems problem because graphs, folds, encodings, and projections recur. **kodama-cpp** consolidates them in one float32 state machine. The

held-out signal and KNN/PLS-LDA classifiers are inherited; grouped proposals, guarded acceptance, and sparse agreement graphs are current policies evaluated separately.

2 KODAMA objective and search

For data $X \in \mathbb{R}^{n \times p}$, labels y , folds Π , and classifier family $\mathcal{F}$, let

$$A(y; X, \mathcal{F}, \Pi) = \frac{1}{n} \sum_{i=1}^n \mathbf{1}[y_i = \hat{y}_i^{(-\Pi(i))}]. \quad (1)$$

KODAMA searches for label vectors with high A ; it does not interpret A as agreement with external truth. The classifier is KNN or SIMPLS plus LDA in the requested feasible latent dimension (de Jong, 1993). Folds remain fixed within a run, and samples sharing a supplied constraint identifier form an atomic proposal group.

Run r uses seed $s + r$ and exactly $L < n$ landmarks. If the request is at least n , the historical $L = \lceil 0.75n \rceil$ rule is retained. Matrix-only input is partitioned into $K_0 = \texttt{splitting}$ coarse k-means strata (100 for $n < 40000$, 300 otherwise). Stratum c , with n_c samples, receives expected quota $q_c = Ln_c/n$: $\lfloor q_c \rfloor$ samples are drawn without replacement and the remaining slots are allocated by randomized systematic rounding of the fractional quotas. This replaces the previous L -center landmark k-means. With auxiliary 2D/3D coordinates, an automatic regular grid uses $\lceil L^{1/d} \rceil$ bins per axis and applies the same quota rule to occupied cells in grid order; no cell is guaranteed a landmark, so isolated spots can be omitted without a density threshold. In both paths, a separate expression-space **splitting** k-means on the selected landmarks preserves the established initial-label construction and keeps sampling strata distinct from labels being optimized. Held-out predictions then propose grouped relabelings whose maximum size decreases smoothly,

$$q_{\max}(t) = 1 + \lfloor (G - 1) [1 - p_t^2(3 - 2p_t)] \rfloor, \quad p_t = \frac{t}{T + 1}, \quad (2)$$

with uniform $q \in \{1, \dots, q_{\max}(t)\}$. A group replacement is sampled from member predictions by multiplicity. The transition matrix $N_{ab} = \#\{i : y_i = a, \hat{y}_i = b\}$ permits absorption only if $N_{ab} - n_a n_b / n > 0$ and $N_{ab} > N_{aa}$; one-class proposals are rejected. PLS-LDA also uses the transition-coarsening budget specified in the supplement.

The proposed vector receives exactly one new CV pass. Let $p_k = n_k/n$, $D = 1 - \sum_k p_k^2$, $H = -\sum_k p_k \log p_k$, $K_{\text{eff}} = e^H$, and $R = \max\{0, \log[K/\max(1, K_{\text{eff}})]\}$. The state-selection score is

$$S(y) = A(y)\sqrt{D} - \mathbf{1}_{\{\mathcal{F}=\text{PLS-LDA}\}}(1 - A(y)) \max\left\{\frac{H}{\log n}, \frac{R}{1 + R}\right\}. \quad (3)$$

Best labels maximize S , with paired raw A returned as **accbest**. Improvements are accepted. Worse proposals have probability $P_{\text{acc}} = \exp[(S_{\text{new}} - S_{\text{cur}})/\tau_t]$. Here $\tau_t = \max\{10^{-9}, 0.10(1 - A_{\text{cur}})(1 - t/T)\}$. The final cycle is greedy up to the floor. Search uses labels and CV predictions, never reference labels.

After T cycles, labels are projected from landmarks. For each original graph edge (i, j) , a_{ij} is the fraction of valid runs agreeing on its endpoints. The contract is $d'_{ij} = (1 + d_{ij})/a_{ij}^2$; zero agreement removes the edge and rows are resorted. Since this is not scale invariant, comparisons fix metric and preprocessing.

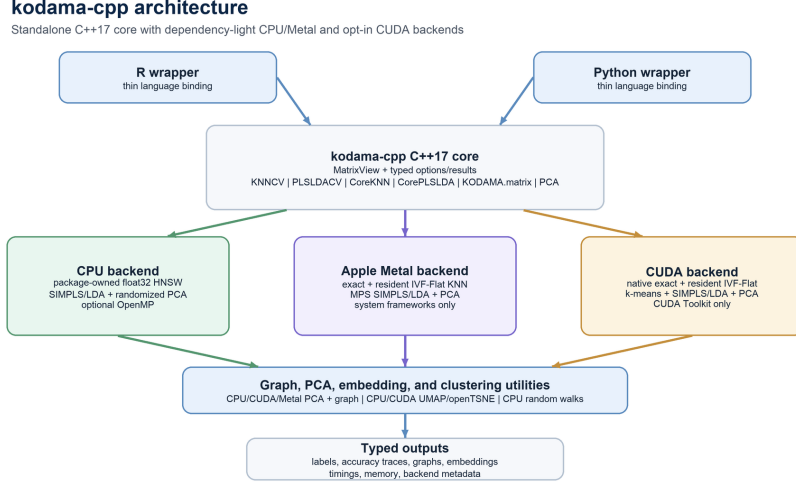


Figure 1: The C++17 core owns KODAMA state and exposes one typed API to R and Python. Backend-specific kernels share the same folds, proposals, component-count semantics, and result contract.

One independent run. Initialize landmarks, labels, folds, and one CV prediction. For $t = 1, \dots, T$, propose grouped/transition moves, evaluate once by CV, update the best-by- S state, and update the current state by cooling. Return the best-by- S labels and their paired raw accuracy. Repeat for M independent seeds, project labels, and reweight the shared graph by agreement.

3 Standalone heterogeneous implementation

Figure 1 shows ownership. CPU provides package-owned HNSW (Malkov and Yashunin, 2020), SIMPLS/LDA, PCA, graph operations, and embeddings. CUDA provides exact/IVF KNN, batched k-means, label-aware SIMPLS/LDA, PCA, and embeddings; Metal provides exact/IVF KNN, k-means, MPS-assisted SIMPLS/LDA, and PCA. For IVF construction on both accelerators, assignments, centroid accumulation and empty-cluster repair, list counting, prefix offsets, and identifier scatter remain on-device. A move-only resident-index API retains the float32 training matrix and trained IVF state across self or external-query calls without a global cache. Unavailable accelerators raise errors instead of falling back.

PLS-LDA computes $X^\top Y$ from class sums, compacts active labels, and reuses fold indices and workspaces. It evaluates the requested feasible component count without internal selection. Results record predictions, folds, confusion matrices, ensembles, resources, parameters, and backend.

For latent training scores $T \in \mathbb{R}^{n \times q}$, class counts n_c , means μ_c , and C active classes, the pooled within-class covariance is

$$\Sigma = \frac{T^\top T - \sum_{c=1}^C n_c \mu_c \mu_c^\top}{\max(1, n - C)}. \quad (4)$$

Table 1: Selected implementation validation. Accuracy is CPU/accelerator; runtime is seconds. The complete KODAMA row reports median raw CV accuracy.

Platform	Dataset/kernel	CPU	Accel.	Speedup	Accuracy
CUDA	MNIST KNN CV	76.769	4.753	16.2	.974/.973
CUDA	MetRef KODAMA, $M = T = 100$	341.648	270.408	1.26	.9924/.9924
CUDA	MetRef PLSLDACV	1.294	0.308	4.20	.992/.993
Metal	MetRef KNN CV	11.145	0.026	425.0	.827/.827
Metal	MetRef PLSLDACV	0.790	0.202	3.90	.991/.990

LDA solves $\Sigma w_c = \mu_c$ by Cholesky and triangular substitution. A fixed scale-normalized ridge sequence advances only after failure; it is a safeguard, not tuning. Prediction uses $\delta_c(t) = t^\top w_c - \frac{1}{2} \mu_c^\top w_c + \log(n_c/n)$; the sequence is in the supplement.

KNN can consume supplied neighbors; graph-only PLS-LDA uses a self-tuning normalized Laplacian and is a spectral surrogate, not data-input parity. Pinned UMAP/openTSNE kernels consume float32 CSR graphs (McInnes et al., 2018; van der Maaten and Hinton, 2008). The R wrapper is at <https://github.com/tkcaccia/kodama-r>; tested Python source is under `split-repos/kodama-python`, pending standalone publication.

4 Validation and scope

CTest and wrappers check numerical, float32, backend, graph, PCA, visualization, and candidate-0.1.0 API contracts. At commit 9da48ee, GitHub Actions passed Linux/macOS CPU, Metal, R/Python, coverage, and documentation jobs. CPU coverage was 63.58% line and 57.03% branch, motivating more state-machine tests. Table 1 reports within-platform measurements.

Five-run MetRef PLSLDACV medians gave 3.90x Metal and 4.20x CUDA speedups over four-thread hosts; accuracy differed by at most one of 873 samples. A matched current-commit $M = T = 100$ run took 341.648/270.408 seconds (1.26x) and 351/730 MB peak host memory. Median raw CV accuracy was .9924 on both; finite precision selected different runs, so label equality is not claimed.

In a deterministic 50,000-by-32 float32 systems microbenchmark with five timed searches after one warm-up, retaining one IVF index reduced 1,000-query rebuild-plus-search time from 0.1179 to 0.0306 seconds on Metal and from 0.0318 to 0.0092 seconds on CUDA (3.86x and 3.45x), with neighbor overlap 1.000. All-row self-search gains were 1.07x and 1.12x because the repeated search itself dominated construction. These timings validate state reuse rather than dataset-level quality.

Matched MetRef KNN ($M = T = 100$) took 610.5 seconds in KODAMA R 2.4.1 and 965.3/235.5/2.36 seconds in kodama-cpp CPU1/CPU4/CUDA. The supplement retains a labeled, non-parity current-R PLS-LDA check.

On one-million-row flow18, exact-quota sampling selected 750,016 landmarks from 300 coarse strata in 0.321 seconds. A CUDA KNN run with $M = 1, T = 10$ took 362.35 seconds and reached raw CV accuracy .9639; 335.08 seconds were spent once on the 100-neighbor global graph, isolating graph construction rather than landmark selection as the remaining large-data bottleneck.

Raising T from 20 to 100 improved median accuracy and fragmentation for USPS and MetRef PLS-LDA, while MetRef KNN remained over-coarsened. At $M = 100$, worst-case agreement-edge standard error is .05; every $M = 50$ graph correlated at least .9888 with $M = 100$. ARI and silhouette changes were nonmonotone.

5 Availability and limitations

The MIT candidate is at <https://github.com/tkcaccia/kodama-cpp>; Metal-derived files retain MIT AND Apache-2.0 notices. Commits 9da48ee/4d5506b are audited core/R baselines; a tag and archive remain pre-submission actions. Repeated CV dominates, M runs are not fully device resident, approximate ties may vary, and CUDA graph $k \leq 256$.

References

- Asa Ben-Hur, Andre Elisseeff, and Isabelle Guyon. A stability based method for discovering structure in clustered data. In *Pacific Symposium on Biocomputing*, pages 6–17, 2002.
- Stefano Cacciatore, Claudio Luchinat, and Leonardo Tenori. Knowledge discovery by accuracy maximization. *Proceedings of the National Academy of Sciences*, 111(14):5117–5122, 2014.
- Stefano Cacciatore, Leonardo Tenori, Claudio Luchinat, Phillip R. Bennett, and David A. MacIntyre. Kodama: an r package for knowledge discovery and data mining. *Bioinformatics*, 33(4):621–623, 2017. doi: 10.1093/bioinformatics/btw705.
- Olivier Chapelle, Bernhard Schölkopf, and Alexander Zien. *Semi-Supervised Learning*. MIT Press, 2006.
- Sijmen de Jong. Simpls: an alternative approach to partial least squares regression. *Chemometrics and Intelligent Laboratory Systems*, 18(3):251–263, 1993.
- Yu. A. Malkov and D. A. Yashunin. Efficient and robust approximate nearest neighbor search using hierarchical navigable small world graphs. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 2020.
- Leland McInnes, John Healy, and James Melville. Umap: Uniform manifold approximation and projection for dimension reduction. *arXiv:1802.03426*, 2018.
- Robert Tibshirani and Guenther Walther. Cluster validation by prediction strength. *Journal of Computational and Graphical Statistics*, 14(3):511–528, 2005.
- Laurens van der Maaten and Geoffrey Hinton. Visualizing data using t-sne. *Journal of Machine Learning Research*, 9:2579–2605, 2008.